



Write, simulate, verify

Write RTL, simulate it, and trust your waveforms

If Chapter 3 taught you how to *think* like hardware, Chapter 4 is where you start *speaking* hardware. Verilog (and its modern superset, SystemVerilog) isn't "software with semicolons." It's a way to describe circuits: wires, registers, timing, and behavior that becomes real silicon later.

Here you'll set up a toolchain, write your first RTL, build a testbench, read waveforms, and learn the top mistakes that make beginners suffer. By the end, you should be able to confidently say: "I wrote it, I simulated it, I verified it." That's the holy trinity.

We'll use an open-source flow that's friendly, fast, and widely used in teaching and hobby/prototyping:

- **Icarus Verilog (iverilog + vvp):** compile + run classic Verilog simulations.
- **GTKWave:** view waveforms (VCD/FST/etc.) like a pro.
- (Optional) **Verilator:** "compiler-style" simulation for speed and stricter checks.

What Verilog is (and what it isn't)

Verilog is a hardware description language

You describe:

- Combinational logic (wires and equations)
- Sequential logic (flip-flops triggered by clock edges)
- Hierarchy (modules connected together)

Verilog is NOT "run line-by-line"

In hardware:

- Many things happen "at once" (concurrently)
- Time matters